

DARTF

Detect Anything in Real Time Faster

W8A8 deployment of the SAM 3 ViT H detector on a Jetson AGX Orin with exact reparametrizations, activation aware GPTQ and block level TensorRT plugins.

Mehmet Kerem Turkcan, 2026

Setting and result

SAM 3 is an open vocabulary detector. Its ViT H backbone has 32 transformer blocks of width 1024 and 5184 tokens per 1008 by 1008 frame, with 4 global and 28 windowed attention blocks.

On a Jetson AGX Orin the FP16 TensorRT engine needs 275 ms per frame. The goal is 8 bit weights and 8 bit activations with detection quality equal to FP32.



All 32 blocks run in INT8, including block 0, which consumes the raw patch embedding and which no earlier recipe could quantize; keeping it in FP16 costs 6 ms and buys nothing measurable.

On the 5000 image COCO val set the INT8 engine scores 56.0 AP against 56.1 for FP32, with every size and recall metric within 0.2 points; on 100 images every detection level count matches within one.

43 percent less time at FP32 quality, with no change to the model, the resolution or the runtime.

Background: uniform quantization and where its error comes from

A symmetric 8 bit quantizer with a single static scale per tensor, which is what TensorRT executes at full speed:

$$Q_s(x) = s \text{clip}(\text{round}(x/s), -127, 127), \quad s = \max|x|/127$$

Rounding error is uniform with variance one twelfth of the step, so the relative error of a tensor is set by its crest factor, the ratio of its largest value to its root mean square:

$$\frac{\|x - Q_s(x)\|}{\|x\|} \approx \frac{s/\sqrt{12}}{\text{rms}(x)} = \frac{\max|x|}{127\sqrt{3} \text{rms}(x)} \approx \frac{\text{crest}(x)}{220}$$

Per token scales would track the range of each token but are not available inside the fused TensorRT kernels; the whole problem is therefore to lower the crest factor of every quantized tensor by exact transformations of the network.

Measured on SAM 3 the crest factor at the LayerNorm fed sites is about 28, close to the square root of the width: a single channel per token carries the whole range, so per tensor INT8 is hopeless there without a change of basis. The proj and fc2 inputs, behind the softmax and the GELU, carry token wise heavy tails instead, which a change of basis cannot fix.

Background: the methods this work builds on

GPTQ rounds weights with second order information. With inputs X it solves column by column, propagating each rounding error to the remaining columns through the inverse Hessian:

$$\min_{\hat{W}} \|(W - \hat{W})X\|_F^2, \quad H = XX^\top$$

SmoothQuant migrates difficulty between an activation and its weight with a per channel scale, which is a diagonal reparametrization:

$$XW = (XS^{-1})(SW), \quad S = \text{diag}(\max |X_{:,k}|^\alpha / \max |W_{k,:}|^{1-\alpha})$$

QuaRot and SpinQuant use a full orthogonal transform instead, folded into the weights, so that outliers are spread over all channels. This is exact only because RMSNorm commutes with rotations:

$$XW = (XQ)(Q^\top W), \quad \text{RMSNorm}(xQ) = \text{RMSNorm}(x)Q$$

Vision transformers use LayerNorm, which subtracts the mean, and the identity above fails. The vision PTQ line, PTQ4ViT, FQ ViT and RepQ ViT, therefore worked with twin uniform quantizers, log quantizers and per channel reparametrizations of the LayerNorm affine, which are all diagonal and cannot mix channels.

What is different here: an exact rotation for LayerNorm

Take any orthogonal Q whose first column is the normalized ones vector, and write y for xQ . Then the mean of x is the first rotated coordinate and the centered norm is the norm of the remaining coordinates:

$$Q^T \mathbf{1} = \sqrt{C} e_0, \quad y_0 = \sqrt{C} \mu, \quad \|x - \mu \mathbf{1}\|^2 = \|y\|^2 - y_0^2$$

LayerNorm in the rotated basis is therefore an RMSNorm over coordinates 1 to C minus 1 with coordinate 0 set to zero, and the affine parameters fold into every consumer:

$$\text{LN}(x) Q = \frac{y - y_0 e_0}{\sigma}, \quad \sigma^2 = \frac{\|y\|^2 - y_0^2}{C} + \epsilon$$
$$(\text{LN}(x) \odot \gamma + \beta) W = \frac{y - y_0 e_0}{\sigma} (Q^T \text{diag}(\gamma) W) + \beta W$$

The residual stream runs rotated end to end: the patch embedding is rotated once, proj and fc2 outputs are rotated on the way in, the neck input is rotated back once. Nothing is computed online at inference; the rewritten network is exact and was verified against PyTorch.

With a Walsh Hadamard Q the crest factor at the LayerNorm fed sites drops from 28 to 4, the value of a Gaussian vector, and block 0, which no rescaling could quantize, becomes an ordinary block. RepQ ViT can only rescale channels; QuaRot and SpinQuant need RMSNorm; this identity needs neither an online operation nor a change of normalization.

What is different here: three more exact rewrites

The attention output is linear in V , so a per head orthogonal R folds into the value projection on the right and into the output projection on the left, as QuaRot does online for language models; here it is folded offline and it lowers the crest factor of the proj input, which no residual stream rotation can reach:

$$a = PV, \quad a(I_H \otimes R) = P(V(I_H \otimes R)) : \quad W_v \leftarrow W_v(I_H \otimes R), \quad W_o \leftarrow (I_H \otimes R)^\top W_o$$

The rotary embedding pairs coordinates and rotates each pair by a position dependent angle. The pairwise rotation of the query is a signed column permutation of the projection weight, which is exact even on INT8 codes, so the rotated branch is just a second projection:

$$\begin{aligned} \text{RoPE}(q) &= \cos \theta \odot q + \sin \theta \odot (qR_{\pi/2}), & qR_{\pi/2} &= x(W_qR_{\pi/2}) \\ R_{\pi/2} &= I_{32} \otimes J, \quad J = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}, & W_qR_{\pi/2} &\text{ is a signed column permutation of } W_q \end{aligned}$$



pairs swap and one of each pair changes sign: exact on INT8 codes

Tokens are partitioned into windows once after the patch embedding and un partitioned once before the neck, with permuted RoPE tables in the global blocks. All 32 blocks become structurally identical, which is what allows one plugin design for the whole trunk.

What is different here: activation aware GPTQ

Standard GPTQ forms its Hessian from full precision inputs and quantizes weights only. Here blocks are quantized in order, each block sees the fake quantized outputs of the quantized prefix, and the activation quantizer of the site is applied before the Hessian is formed, so the weight rounding compensates the activation rounding it will meet:

$$\hat{X} = Q_{s_a}(\tilde{X}), \quad \hat{W} = \text{GPTQ}(W, H = \hat{X}\hat{X}^\top), \quad b \leftarrow b - \mathbb{E}_t[\hat{X}\hat{W}^\top - \tilde{X}W^\top]$$

The tilde marks inputs propagated through the quantized prefix; the expectation runs over calibration tokens. Scales, codes and biases are consumed verbatim by the ONNX quantizer, so the deployed graph is the one the Hessians saw.

The one site neither rotation reaches is the fc2 input behind the GELU. There the SmoothQuant migration is exact and free, because the plugin applies the per channel factor inside the fc1 requantization epilogue:

$$x_k / c_k, \quad W_2[k, :] c_k, \quad c_k = \max |x_k|^{1/2} / \max |W_2[k, :]|^{1/2}$$



Feature error relative to plain GPTQ, in percent. Each step is free at inference: the same engine, the same latency.

The crest factor argument, measured



after the rotation: crest factor 4.07, the value of a Gaussian vector

A learned rotation, trained on a crest factor surrogate, lowers the crest factor further but leaves the network error unchanged: once any rotation is applied, the LayerNorm fed sites have left the error budget. The Hadamard is enough.

Per site ablations agree: after the rotations most of the remaining error sits behind the softmax and the GELU, where no residual stream rotation reaches. That is what the per head V rotation and the fc2 channel scales address.

Feature error stops predicting detection quality once it is small: engines that differ by a third in feature error are indistinguishable at detection level. Quality has to be judged on detections, against a measured noise floor.

Making TensorRT execute it: block level plugins

TensorRT explicit quantization has two rules that shape the design. A plugin is not handed the scale of an INT8 input that a QuantizeLinear node produces, and a plugin may not produce the INT8 tensor that a DequantizeLinear node consumes.

A plugin that replaces a single fused kernel therefore becomes an island: the surrounding quantize and dequantize nodes lose their fusions. Only plugins that remove work or internalize whole subgraphs pay.



three plugins per block; INT8 tensors never cross a plugin boundary, only the fp16 residual stream does

Each block became three plugins on fp16 residual stream edges, with CUTLASS INT8 GEMMs, RoPE and GELU in the epilogues and the residual add fused. TensorRT keeps the graph and the scheduling; the plugins own everything between two residual adds. That alone took the trunk from 187 to 166 ms.

A pipelined flash attention kernel for Ampere

Flash attention keeps a running maximum and normalizer per row and never materializes the score matrix:

$$S_t = QK_t^\top, \quad m_t = \max(m_{t-1}, \text{rowmax } S_t), \quad P_t = e^{S_t - m_t}$$
$$\ell_t = e^{m_{t-1} - m_t} \ell_{t-1} + \text{rowsum } P_t, \quad O_t = e^{m_{t-1} - m_t} O_{t-1} + P_t V_t, \quad O = O_T / \ell_T$$

The first kernel ran at less than half of the tensor pipe floor. Two measurements explained the gap: an ablation that compiled out one component at a time showed that the tensor work and the FP32 softmax work simply add, and a SASS histogram showed the loop dominated by address arithmetic rather than tensor instructions.

tensor pipe



FP32 softmax pipe



pipelined: the score tile of step t plus 1 is issued before the softmax of step t

The fix issues the score tile of step t plus 1 on the tensor pipe before the softmax of step t runs on the FP32 pipe, with the scores double buffered in registers, and hoists the shared memory addressing out of the loop. Global attention got 19 percent faster and windowed attention 12 percent, ahead of TensorRT on the same GPU, exact to fp16 rounding.

The grounding head: exact sharing across prompts

Scoring a prompt is linear in the number of prompts, and half of it is global attention over the image tokens. Kernel fusion does not help there; TensorRT is already at parity. The lever is structure, not kernels.

The first encoder layer depends only on the image. It is computed once per image by a prefix engine and broadcast to every prompt of a class bucket by the post engine; CUDA graph replay removes the launch overhead of the rest.



The outputs are bit identical: the head is faster without a single approximation.

What is new, in five sentences

A closed form rotation for LayerNorm networks: any orthogonal transform that sends the ones direction to one coordinate turns LayerNorm into a masked RMSNorm, so rotation based quantization applies to vision transformers without online operations and without changing the normalization.

A per head rotation of the value subspace and a signed permutation form of the rotary embedding, both folded into the weights, extend the exactness to the attention output and to the rotary branch.

An activation aware sequential GPTQ with bias correction whose Hessians come from the quantized inputs the deployed graph will see, plus a per channel migration at the one site rotation cannot reach.

A deployment recipe for explicit quantization in TensorRT: whole block plugins with INT8 kept inside, and a token major flash attention kernel that is faster than the vendor kernel on Ampere by overlapping its tensor and softmax pipes.

A validation methodology at detection level with a measured noise floor, which shows that feature level error stops predicting detection quality once it is small, so the last rounding decisions must be judged on detections.

Built on

GPTQ: Frantar et al. 2023; SmoothQuant: Xiao et al. 2024

QuaRot: Ashkboos et al. 2024; SpinQuant: Liu et al. 2025

FlashAttention 2: Dao 2023; FlashAttention: Dao et al. 2022

BRECQ: Li et al. 2021; AdaRound: Nagel et al. 2020

RepQ ViT: Li et al. 2023; FQ ViT: Lin et al. 2023

PTQ4ViT: Yuan et al. 2024; FlatQuant: Sun et al. 2025

DuQuant: Lin et al. 2024; Massive activations: Sun et al. 2024

Quantizable Transformers: Bondarenko et al. 2023; LLM int8: Dettmers et al. 2022

SAM 2: Ravi et al. 2024; Segment Anything: Kirillov et al. 2023

ViT: Dosovitskiy et al. 2021; RoPE: Su et al. 2023

ViTDet: Li et al. 2022; SAM 3: Carion et al. 2026

DART: Turkcan 2026

Summary

Exact rewrites: rotated residual stream with a masked RMSNorm, per head V rotation, RoPE fold and window major layout.

Activation aware GPTQ with bias correction and per channel fc2 scales.

Block level TensorRT plugins and a pipelined attention kernel; exact prefix sharing in the head.

275 ms to 158 ms per frame and 42 percent less energy, at FP32 detection quality.

Code and the full pipeline: the dartf folder of the DART repository.